

PHÁT HIỆN LỖ HỔNG TRONG CHỨC NĂNG ĐĂNG NHẬP

Phân tích bảo mật và xây dựng phiên bản đăng nhập an toàn

1. Phân tích các vấn đề bảo mật

Vấn đề	Nguy cơ	Cách khắc phục
Mật khẩu được so sánh trực tiếp	<code>data.password != user.password</code> cho thấy mật khẩu có khả năng được lưu dưới dạng plaintext. Nếu database bị lộ, mật khẩu người dùng có thể bị đọc trực tiếp.	Không lưu mật khẩu gốc. Sử dụng Bcrypt để hash mật khẩu và dùng <code>bcrypt.checkpw()</code> để kiểm tra khi đăng nhập.
Có thể đang lưu mật khẩu dạng plaintext	Người có quyền truy cập database có thể xem được mật khẩu thật. Nếu người dùng sử dụng lại mật khẩu ở dịch vụ khác, hậu quả có thể nghiêm trọng hơn.	Lưu <code>password_hash</code> thay vì <code>password</code> . Chỉ lưu chuỗi hash Bcrypt trong database.
Đưa password vào JWT Payload	Đoạn <code>"password": user.password</code> đưa mật khẩu vào token. JWT thường chỉ được ký, không phải mã hóa, nên payload có thể bị đọc nếu token bị lấy được.	Tuyệt đối không đưa password hoặc <code>password_hash</code> vào JWT. Chỉ lưu thông tin cần thiết như <code>sub</code> , <code>role</code> và <code>exp</code> .
Secret key JWT bị hard-code	Secret key <code>"123456"</code> quá yếu. Nếu bị lộ, kẻ tấn công có thể tạo JWT giả và giả mạo người dùng.	Dùng secret key mạnh, ngẫu nhiên và lưu trong biến môi trường <code>.env</code> ; không hard-code trong source code.
Thông báo lỗi quá chi tiết	Thông báo <code>"Email không tồn tại"</code> cho phép attacker biết email nào đã đăng ký, từ đó thực hiện user enumeration.	Dùng thông báo chung như <code>"Email hoặc mật khẩu không chính xác"</code> .
Không kiểm tra tài khoản bị khóa	Tài khoản đã bị vô hiệu hóa vẫn có thể đăng nhập nếu email và mật khẩu đúng.	Kiểm tra <code>user.is_active</code> trước khi tạo access token.
JWT không có thời gian hết hạn	Token được tạo mà không có <code>exp</code> có thể được sử dụng vô thời hạn. Nếu token bị đánh cắp, attacker có thể sử dụng lâu dài.	Thêm trường <code>exp</code> vào JWT, ví dụ thời hạn 30 phút.
JWT chứa quá nhiều thông tin	Đưa email, password và role vào token là không cần thiết, đặc biệt password có nguy cơ bị lộ.	Chỉ đưa thông tin tối thiểu cần thiết vào JWT, ví dụ <code>sub</code> , <code>role</code> và <code>exp</code> .
Không truy vấn lại database khi xác thực	Nếu hệ thống chỉ tin vào dữ liệu trong JWT thì user đã bị khóa hoặc xóa vẫn có thể tiếp tục sử dụng token cũ.	Lấy <code>user_id</code> từ JWT, sau đó query lại database để kiểm tra user tồn tại và còn hoạt động.

2. Phiên bản code đăng nhập an toàn hơn

2.1. Model User

```
from sqlalchemy import Column, Integer, String, Boolean
from app.database import Base
```

```
class User(Base):
```

```

__tablename__ = "users"

id = Column(Integer, primary_key=True, index=True)

email = Column(
    String(100),
    unique=True,
    nullable=False,
    index=True
)

password_hash = Column(
    String(255),
    nullable=False
)

full_name = Column(
    String(100),
    nullable=False
)

role = Column(
    String(20),
    default="student",
    nullable=False
)

is_active = Column(
    Boolean,
    default=True,
    nullable=False
)

```

2.2. Hash và kiểm tra mật khẩu bằng Bcrypt

```

import bcrypt

def hash_password(password: str) -> str:
    hashed = bcrypt.hashpw(
        password.encode("utf-8"),
        bcrypt.gensalt()
    )
    return hashed.decode("utf-8")

def verify_password(
    password: str,
    password_hash: str
) -> bool:
    return bcrypt.checkpw(
        password.encode("utf-8"),
        password_hash.encode("utf-8")
    )

```

2.3. Tạo JWT có thời hạn

```

import os
import jwt

```

```

from datetime import datetime, timedelta, timezone
from dotenv import load_dotenv

load_dotenv()

JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY")
JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
ACCESS_TOKEN_EXPIRE_MINUTES = int(
    os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES", 30)
)

def create_access_token(user):
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )

    payload = {
        "sub": str(user.id),
        "role": user.role,
        "exp": expire
    }

    token = jwt.encode(
        payload,
        JWT_SECRET_KEY,
        algorithm=JWT_ALGORITHM
    )

    return token

```

2.4. API Login

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from app.database import get_db
from app.models.user import User
from app.security import (
    verify_password,
    create_access_token,
    ACCESS_TOKEN_EXPIRE_MINUTES
)

router = APIRouter(
    prefix="/auth",
    tags=["Authentication"]
)

@router.post("/login")
def login(
    data: LoginRequest,
    db: Session = Depends(get_db)
):
    # 1. Tìm user theo email
    user = db.query(User).filter(
        User.email == data.email
    ).first()

    # 2. Không tiết lộ email có tồn tại hay không

```

```

if user is None:
    raise HTTPException(
        status_code=401,
        detail="Email hoặc mật khẩu không chính xác"
    )

# 3. Kiểm tra tài khoản đang hoạt động
if not user.is_active:
    raise HTTPException(
        status_code=401,
        detail="Tài khoản đã bị khóa"
    )

# 4. Kiểm tra password bằng Bcrypt
if not verify_password(
    data.password,
    user.password_hash
):
    raise HTTPException(
        status_code=401,
        detail="Email hoặc mật khẩu không chính xác"
    )

# 5. Tạo JWT
access_token = create_access_token(user)

# 6. Trả token
return {
    "success": True,
    "access_token": access_token,
    "token_type": "bearer",
    "expires_in": ACCESS_TOKEN_EXPIRE_MINUTES * 60
}

```

3. Luồng xử lý đăng nhập sau khi sửa

1. Client gửi email và password đến POST /auth/login.
2. Server tìm người dùng trong database theo email.
3. Nếu không tồn tại, trả 401 với thông báo chung: Email hoặc mật khẩu không chính xác.
4. Nếu tài khoản tồn tại, kiểm tra is_active. Nếu tài khoản bị khóa, từ chối đăng nhập.
5. Server dùng Bcrypt để so khớp password người dùng nhập với password_hash trong database.
6. Nếu password sai, trả 401 với thông báo chung.
7. Nếu password đúng, server tạo JWT.
8. JWT chỉ chứa thông tin cần thiết như user ID (sub), role và thời gian hết hạn exp.
9. Server trả access_token, token_type và expires_in cho client.

Sơ đồ:

Client → POST /auth/login → Tìm User → Kiểm tra is_active → Bcrypt verify password → Tạo JWT → Trả Access Token

4. Các điểm cần lưu ý

- Không lưu password dạng plaintext.
- Không đưa password hoặc password_hash vào JWT.
- Không sử dụng secret key yếu hoặc hard-code secret key.
- JWT cần có thời gian hết hạn, ví dụ 30 phút.
- Dùng thông báo lỗi chung để hạn chế user enumeration.
- Kiểm tra trạng thái tài khoản trước khi cấp token.
- Khi gọi /auth/me, cần kiểm tra chữ ký và thời hạn JWT rồi truy vấn lại database theo user_id.

5. Kết luận

Đoạn code đăng nhập ban đầu có nhiều vấn đề bảo mật, đặc biệt là lưu và so sánh mật khẩu trực tiếp, đưa mật khẩu vào JWT, sử dụng secret key yếu, không đặt thời hạn cho token và tiết lộ email có tồn tại hay không. Phiên bản đã sửa sử dụng Bcrypt để hash và kiểm tra mật khẩu, JWT chỉ chứa những thông tin cần thiết, secret key được bảo vệ bằng biến môi trường, token có thời hạn và hệ thống kiểm tra trạng thái tài khoản. Các biện pháp này giúp hạn chế nguy cơ lộ mật khẩu, giả mạo token và dò tìm tài khoản.